FCM 1: Assignment 1

Dr. Gallivan

Alex Sampson

Contents

1 Executive Summary **3**

2 Statement of the Problem **3**

3 Mathematical Description **3**
3.1 $z = L\,v$ (dense, column-oriented) . . . 3
3.2 $z = L\,v$ (compressed-column, 1D) . . . 4
3.3 $z = L\,v$ (banded with two subdiagonals) . . . 5
3.4 $A = L\,U$ (outer product from stored M) . . . 5

4 Implemented Subroutines **7**
4.1 `Lvmult_col.m` (unit lower, 2-D array) . . . 7
4.2 `Lvmult_compcol.m` (unit lower, compressed columns) . . . 7
4.3 `Lvmult_banded.m` (unit lower, two subdiagonals) . . . 8
4.4 `LUmult_outer.m` (outer-product, stored L/U in M) . . . 9

5 Driver and Experiments **9**

6 Results and Discussion **10**

7 Conclusions **11**

8 Disclaimer **11**

A Figures **12**
A.1 Lv (unit lower) routines . . . 12
A.2 Uv (given for validation) . . . 14

B Driver Script **15**

1 Executive Summary

In this project we develop and validate numerical schemes for structured linear algebra operations. Specifically, we implement subroutines for computing $z = Lv$ where L is a randomly generated unit lower triangular matrix and v a random vector, as well as $A = LU$ where L and U are stored together in a two-dimensional array M. Each routine exploits structure in a different way: a standard column-oriented form, a compressed-column 1D representation, a banded format with two subdiagonals, and an M that stores both L and U for efficient LU multiplication.

Experimental validation uses test problems with $n \in [30, 900]$ (step size $\Delta n = 10$). For each n, we compute z_{comp} or A_{comp} using our subroutines and compare against the MATLAB "true" products $z_{\text{true}} = Lv$ and $A_{\text{true}} = LU$ in both absolute and relative ℓ_2 norms:

$$z_{\text{err}} = z_{\text{true}} - z_{\text{comp}}, \qquad A_{\text{err}} = A_{\text{true}} - A_{\text{comp}}.$$

Errors $\|z_{\text{err}}\|_2$ and $\|A_{\text{err}}\|_2$ remained on the order of machine precision, with maximum relative errors typically 10^{-15} to 10^{-16} across all tested dimensions. This indicates our implementations closely match MATLAB's built-in routines.

In addition to accuracy, we also consider efficiency. Our routines demonstrate the expected scaling behavior: the dense and compressed-column schemes require quadratic work, while the banded scheme achieves linear cost by exploiting its two-subdiagonal structure. The storeed M format supports LU multiplication in cubic time while maintaining quadratic storage. A full breakdown of flop counts and storage requirements is given in the Mathematical Description section. These results confirm that the implementations are numerically stable and exploit matrix structure effectively.

2 Statement of the Problem

The goal of this project is to implement and validate numerical routines for structured matrix-vector and matrix-matrix products. Specifically, we consider products of the form $z = Lv$, where L is a unit lower triangular matrix, and $A = LU$, where L and U are respectively unit lower and upper triangular. Several storage formats are used to exploit structure, including standard 2-D arrays, compressed 1-D column storage, and banded storage with two subdiagonals. The combined LU product is implemented in a single storage 2-D array. Each routine must be tested against MATLAB's built-in operations for a range of matrix sizes, with correctness assessed through relative and absolute errors in appropriate vector and matrix norms.

3 Mathematical Description

In this section we describe the mathematical processes carried out by each routine. We illustrate the updates with small examples that make the storage format and the sequence of operations explicit. In each case we compare the computed result with the exact product to confirm correctness, and we state the complexity in terms of time ($O(\cdot)$ flops) and space ($O(\cdot)$ storage). We count one multiply and one add per update of the form $z_i \leftarrow z_i + a\,b$ or $A_{ij} \leftarrow A_{ij} + a\,b$.

3.1 $z = L\,v$ (dense, column-oriented)

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ \ell_{21} & 1 & 0 & 0 \\ \ell_{31} & \ell_{32} & 1 & 0 \\ \ell_{41} & \ell_{42} & \ell_{43} & 1 \end{bmatrix}, \qquad v = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}, \qquad z \leftarrow v.$$

Process by columns ($j = 1, 2, 3$):

Column 1 (update rows $i = 2, 3, 4$):

$$z_2 \leftarrow z_2 + \ell_{21} v_1, \quad z_3 \leftarrow z_3 + \ell_{31} v_1, \quad z_4 \leftarrow z_4 + \ell_{41} v_1.$$

Column 2 (update rows $i = 3, 4$):

$$z_3 \leftarrow z_3 + \ell_{32} v_2, \quad z_4 \leftarrow z_4 + \ell_{42} v_2.$$

Column 3 (update row $i = 4$):

$$z_4 \leftarrow z_4 + \ell_{43} v_3.$$

Final:

$$z = \begin{bmatrix} v_1 \\ v_2 + \ell_{21} v_1 \\ v_3 + \ell_{31} v_1 + \ell_{32} v_2 \\ v_4 + \ell_{41} v_1 + \ell_{42} v_2 + \ell_{43} v_3 \end{bmatrix}.$$

Flops. Column j performs $(n - j)$ updates; each update is 1 multiply + 1 add.

$$\#\text{updates} = \sum_{j=1}^{n-1} (n - j) = \frac{n(n-1)}{2}.$$

We have $\frac{n(n-1)}{2}$ multiplies + $\frac{n(n-1)}{2}$ adds. So in general the time complexity is: $O(n^2)$.

Storage. If L is stored as a full $n \times n$ array: n^2 reals. If only the lower triangle (incl. ones) is stored: $\frac{n(n+1)}{2}$. Either way its storage complexity is: $O(n^2)$.

3.2 $z = L\,v$ (compressed-column, 1D)

Store only the *strict* lower triangle by columns in a 1D array

$$L_c = \left[\, \ell_{21},\ \ell_{31},\ \ell_{41} \ \mid\ \ell_{32},\ \ell_{42} \ \mid\ \ell_{43} \,\right]^\top.$$

Process entries of L_c in this order while applying the same updates as above. The result matches the dense case exactly.

Flops. Identical arithmetic work to the dense Lv:

$$\#\text{updates} = \frac{n(n-1)}{2} \quad \Rightarrow \quad O(n^2) \text{ flops.}$$

Storage. Only strict lower entries are stored:

$$\#\text{stored} = \frac{n(n-1)}{2}.$$

So the storage complexity is still: $O(n^2)$

3.3 $z = L\,v$ (banded with two subdiagonals)

Assume $L = I + \mathrm{diag}(Lb1, -1) + \mathrm{diag}(Lb2, -2)$ with

$$Lb1 = \begin{bmatrix} a \\ b \\ c \end{bmatrix}, \qquad Lb2 = \begin{bmatrix} p \\ q \end{bmatrix}.$$

Then, starting with $z \leftarrow v$:

- **First subdiag:** $z_2 \leftarrow z_2 + a\,v_1, \quad z_3 \leftarrow z_3 + b\,v_2, \quad z_4 \leftarrow z_4 + c\,v_3$

- **Second subdiag:** $z_3 \leftarrow z_3 + p\,v_1, \quad z_4 \leftarrow z_4 + q\,v_2$

so

$$z = \begin{bmatrix} v_1 \\ v_2 + av_1 \\ v_3 + bv_2 + pv_1 \\ v_4 + cv_3 + qv_2 \end{bmatrix}.$$

Flops. There are $(n-1)$ updates from the first subdiagonal and $(n-2)$ from the second:

$$\#\text{updates} = (n-1) + (n-2) = 2n - 3.$$

We have $(2n-3)$ multiplies + $(2n-3)$ adds. So in general our time complexity is $O(n)$ flops.

Storage. Store $Lb_1 \in \mathbb{R}^{n-1}$ and $Lb_2 \in \mathbb{R}^{n-2}$ (diagonal ones are implicit):

$$\#\text{stored} = (n-1) + (n-2) = 2n - 3.$$

So our space complexity is: $O(n)$.

3.4 $A = L\,U$ (outer product from stored M)

We store L and U into a single $n \times n$ array M:

$$M_{ij} = \begin{cases} U_{ij}, & i \le j, \\ L_{ij}, & i > j, \end{cases} \qquad L_{ii} = 1 \text{ (implicit)}.$$

For $n = 4$, the storage array M has the form

$$M = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ \ell_{21} & u_{22} & u_{23} & u_{24} \\ \ell_{31} & \ell_{32} & u_{33} & u_{34} \\ \ell_{41} & \ell_{42} & \ell_{43} & u_{44} \end{bmatrix} = \begin{bmatrix} | & | & | & | \\ L_1 & L_2 & L_3 & L_4 \\ | & | & | & | \end{bmatrix} = \begin{bmatrix} - & U_1 & - \\ - & U_2 & - \\ - & U_3 & - \end{bmatrix}.$$

To form $A = L\,U$, we add outer products column by column:

$$A \;=\; \sum_{k=1}^{n} L_k\,U_k.$$

For instance, at $k = 1$,

$$L_1 = \begin{bmatrix} 1 \\ \ell_{21} \\ \ell_{31} \\ \ell_{41} \end{bmatrix}, \qquad U_1 = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \end{bmatrix},$$

so $A^{(1)} = L_1 U_1$ contributes the first rank-one block. Subsequent k add smaller trailing blocks. The sum reproduces LU without ever unpacking M.

Complexity. At step k, $(n-k+1)^2$ entries are updated. Total updates:

$$\sum_{k=1}^{n} (n-k+1)^2 = \frac{n(n+1)(2n+1)}{6}.$$

Each requires 1 multiply + 1 add, therefor our time complexity is $O(n^3)$ flops.

Storage: M uses n^2 entries (strict lower of L plus upper of U) and the result A is also n^2. So the storage complexity is: $O(n^2)$.

Table 1: Time (flops) and space (storage) complexity of routines.

Routine	**Time Complexity**	**Space Complexity**
`Lvmult_col` (dense column-oriented)	$O(n^2)$	$O(n^2)$
`Lvmult_compcol` (compressed column)	$O(n^2)$	$O(n^2)$
`Lvmult_banded` (two subdiagonals)	$O(n)$	$O(n)$
`LUmult_outer` (LU stored in M)	$O(n^3)$	$O(n^2)$

Error calculation

For each test we form a MATLAB "true" result and a computed result from our routine, then measure the discrepancy.
Let $z_{\text{true}} = Lv$ be computed by MATLAB and z_{comp} by our subroutine. Define

$$z_{\text{err}} = z_{\text{true}} - z_{\text{comp}},$$

Here $\|z_{\text{err}}\|_2$ is calculated using the standard ℓ_2 norm. The relative error is then:

$$\text{RelErr}(z) \;=\; \frac{\|z_{\text{err}}\|_2}{\|z_{\text{true}}\|_2}.$$

Let $A_{\text{true}} = LU$ be computed by MATLAB and A_{comp} by our Storage-M routine. Define

$$A_{\text{err}} = A_{\text{true}} - A_{\text{comp}},$$

Where $\|A_{\text{err}}\|_2$ is given by

$$\|A\|_2 := \max_{\|x\|_2=1} \|Ax\|_2.$$

The relative error is then again,

$$\text{RelErr}(A) \;=\; \frac{\|A_{\text{err}}\|_2}{\|A_{\text{true}}\|_2}.$$

Remark on ℓ_∞ norms: While not done in this report due to time constraints, it would also be useful to potentially evaluate errors in the other norms. For instance the ℓ_∞ norm, $\|z_{\text{err}}\|_\infty$ and $\|A_{\text{err}}\|_\infty$ respectively.

The ℓ_2 norm measures the overall magnitude of error across all entries, but it can average out a single large discrepancy if the rest of the vector or matrix is accurate. In contrast, the ℓ_∞ norm highlights the worst-case error in any component. Together, these norms give a more complete picture of stability: ℓ_2 captures global accuracy, while ℓ_∞ detects localized outliers that might be hidden in the average.

4 Implemented Subroutines

4.1 `Lvmult_col.m` (unit lower, 2-D array)

This routine computes $z \leftarrow Lv$ where L is a unit lower triangular matrix stored in a standard $n \times n$ dense array. The diagonal entries are implicit ones, so only the strict lower part of M is referenced during updates. The implementation proceeds column by column, adding the contribution of each column of L to the trailing part of z. Its cost is $O(n^2)$ operations and storage $O(n^2)$.

```
function [z]=Lvmult_col(M,v,z,n)

%A function that takes in arguments:
% M -- The UNIT Lower triangular matrix doing the multiplication
% v -- The vector that we are operating on
% z -- a vector initialized as v, that we will update at each step
% of a loop -- again, we technically dont need this as an argument, its
% defined inside the function, it will just stay for driver compatability
% purposes.

% n -- the dimension of our matrix/vector's (M is square since it's lower
% triangular)



  Mrdim=size(M,1); % Tells us how many rows M has
  Mcdim=size(M,2); % Tells us how many collumns M has
                   % If M is square, Mcdim=Mrdim
  zdim=size(z,1);  % Tells us the length of z, note z is stored as a row vector
  vdim=size(v,1);  % Tells us the length of v, note v is stored as a row vector



  z = v(1:n);                                  %Initializes z as v
for j = 1:n-1
  z(j+1:n) = z(j+1:n) + M(j+1:n, j) * v(j);    %updates the 'tails' of z at each step by
end                                            %adding the next col mult. of L with v
```

4.2 `Lvmult_compcol.m` (unit lower, compressed columns)

Here the same product $z \leftarrow Lv$ is implemented using a compressed 1-D column storage scheme. The strict lower triangle of L is stored sequentially into a vector `Lc`, reducing storage roughly by half compared with the dense representation. The diagonal is still implicit. The routine loops over columns, accumulating contributions from each stored segment of `Lc`.

```
function [z] = Lvmult_compcol(Lc, v, z, n)
% Lvmult_compcol  Compute w = L*v where L is unit lower triangular,
% stored in compressed-column 1D form (strict lower part).
% Lc packs column j entries for rows i=j+1..n.

  v = v(:);        % ensure column
  z = v;           % diagonal ones contribution
  k = 1;           % Initialize Index

  for j = 1:n-1
    xj = v(j);                  % Pick out entries of v at each step.
    for i = j+1:n               % multiplies the associated entry of Lc by the appropriate section of v.
      z(i) = z(i) + Lc(k) * xj; % rewrites z at each step.
      k = k + 1;
    end
  end
end
```

4.3 Lvmult_banded.m (unit lower, two subdiagonals)

This variant assumes L has exactly two nonzero subdiagonals, stored in 1-D arrays `Lb1` and `Lb2`. The routine first initializes $z = v$ (the diagonal contribution) and then adds in the first and second subdiagonal terms. Because the bandwidth is fixed, the cost is only $O(n)$ operations, much more efficient for large n.

```
function [z] = Lvmult_banded(Lb1, Lb2, v, z, n)
% Lvmult_banded  Compute z = L*v for unit-lower L with exactly two
% lower off-diagonals stored as 1D arrays:
%   Lb1(j) = L(j+1, j),  j = 1..n-1   (first subdiagonal)
%   Lb2(j) = L(j+2, j),  j = 1..n-2   (second subdiagonal)
% Diagonal 1's are implicit.
%
% Inputs:
%   Lb1 : (n-1)-by-1
%   Lb2 : (n-2)-by-1
%   v   : n-by-1
%   z   : n-by-1 Initialized as zeros to store and overwrite
%   n   : dimension


    z = v; % Start with the diagonal contribution


  %----------------- First subdiagonal: rows j+1 get Lb1(j)*v(j)----------
  for j = 1:n-1
    z(j+1) = z(j+1) + Lb1(j) * v(j);
  end
  %-----------------------------------------------------------------------
  % --------------- Second subdiagonal: rows j+2 get Lb2(j)*v(j)----------
  for j = 1:n-2
    z(j+2) = z(j+2) + Lb2(j) * v(j);
  end
  %-----------------------------------------------------------------------
  return
end
```

4.4 LUmult_outer.m (outer-product, stored L/U in M)

This routine multiplies L and U directly from a singl storage array `M` that contains the strict lower triangle of L and the diagonal + upper triangle of U. The algorithm uses the outer-product formulation

$$A = \sum_{k=1}^{n} L_k U_k,$$

but only traverses the nonzero regions permitted by the triangular structure. It avoids explicitly forming separate L and U matrices.

```
function [Acomp] = LUmult_outer(M, Acomp, n)
%
% Compute Acomp = L*U from storage M, without unpacking.
% Storing in M:
%   - i > j : M(i,j) = L(i,j)          (strict lower of L)
%   - i <= j: M(i,j) = U(i,j)          (diag + upper of U)
%   - L has unit diagonal: L(i,i) = 1  (implicit, not stored)
%
% Outer-product scheme:
%   For k = 1..n, add L(:,k)*U(k,:) but only where entries can be nonzero:
%     rows i = k..n  (since L(i,k)=0 for i<k; L(k,k)=1)
%     cols j = k..n  (since U(k,j)=0 for j<k)


  Acomp=zeros(n); %Initializes A as an n-by-nx of zeros.

  for k = 1:n     % iterate rows where L(i,k) may be nonzero
    for i = k:n   % L(i,k): 1 on the diagonal, otherwise from strict-lower of M
      if i == k   % diag elements of L
        l = 1;
      else
        l = M(i,k);      % L(i,k) from M
      end

      if l ~= 0          % iterate cols where U(k,j) may be nonzero
        for j = k:n      % U(k,j) sits in the upper/diag part of M
          u = M(k,j);    % U(k,j) from storage M
          if u ~= 0
            Acomp(i,j) = Acomp(i,j) + l * u;
          end
        end
      end
    end
  end

  return
end
```

5 Driver and Experiments

To support the new storage formats, I extended the driver in four places:

(i) routine selection

(ii) construction of test data in the required stored formats

(iii) formation of the "true" answer for validation, and

(iv) dispatch to the correct subroutine.

(1) Routine selector: I added `routinenum = 7` (Lv with compressed-column storage) and `routinenum = 8` (Lv with banded storage), together with corresponding header messages in the printout block. This lets the same driver exercise the new formats.

(2) Building the storage inputs: After generating dense $L = \text{eye}(n)+\text{tril}(\text{Temp}, -1)$ and $U = \text{triu}(\text{Temp})$ for a given n:

- **Compressed column (`Lc`).** I construct a length $n(n-1)/2$ vector `Lc` that stores the strict lower part of L column by column. The inner loop runs `for j=1:n-1` and writes `Lc(k)=L(i,j)` for `i=j+1:n` while incrementing the index `k`. The unit diagonal of L is implicit, so no diagonal entries are stored.
- **Banded (`Lb1, Lb2`).** I extract the two subdiagonals with `Lb1 = diag(L,-1)` (length $n-1$) and `Lb2 = diag(L,-2)` (length $n-2$). For clarity and for computing the true result (see below), I also form a banded copy L_B that has ones on the diagonal and only these two subdiagonals populated from `Lb1` and `Lb2`.

(3) True results: When validating:

- For `routinenum==7` (compressed column), I keep $z_{\text{true}} = L\,v$.
- For `routinenum==8` (banded), I use only the two stored bands by computing $z_{\text{true}} = L_B\,v$. This guarantees we validate against exactly the structure the banded routine sees (unit diagonal + two subdiagonals), not the full L.

(4) Subroutine calls. In the execution switch:

- `routinenum==7` calls `Lvmult_compcol(Lc, v, zcomp, n)`.
- `routinenum==8` calls `Lvmult_banded(Lb1, Lb2, v, zcomp, n)`.

After each call, the driver computes the error vector $z_{\text{err}} = z_{\text{comp}} - z_{\text{true}}$, records $\|z_{\text{err}}\|_2$, $\|z_{\text{true}}\|_2$, and the relative error $\|z_{\text{err}}\|_2/\|z_{\text{true}}\|_2$, and accumulates the per-n mean and max for plotting.

Notes on reproducibility and plots: The driver saves the RNG state before building each random instance (useful for replaying a specific failing case). Output mode (3) produces the mean/max relative error vs. n plots

Why these changes matter: storing L into `Lc` removes all forbidden zeros and the implicit unit diagonal, so the compressed-column routine touches exactly the operations that should exist mathematically. Likewise, validating the banded routine against L_B (not the full L) isolates the intended $O(n)$ stencil action of the two bands and avoids attributing missing far-off diagonals as "errors."

6 Results and Discussion

To evaluate the correctness and stability of the implemented routines, we computed both the mean and maximum relative errors over multiple trials for a sweep of matrix sizes. Again, the relative error being measured via:

$$\text{RelErr}_2 = \frac{\|z_{\text{true}} - z_{\text{comp}}\|_2}{\|z_{\text{true}}\|_2},$$

and,

$$\text{RelErr}_2 = \frac{\|A_{\text{true}} - A_{\text{comp}}\|_2}{\|A_{\text{true}}\|_2}.$$

Across all routines tested (Lv with 2-D storage, compressed column, and banded storage, and LU multiplication with storage array M), the observed relative errors were on the order of 10^{-15} to 10^{-16}. Meaning our computed z and A agree very closely with the routines given in MATLAB for Lv and LU multiplication. The figures below **(See Appendix A)** summarize the mean and maximum relative errors as functions of n for each routine and data distribution (normal vs. uniform). The flat behavior of the error curves across problem sizes confirms the routines are numerically stable and that their accuracy is limited only by machine precision.

7 Conclusions

In this project, we successfully implemented and tested several structured matrix–vector and matrix–matrix multiplication routines, focusing on unit lower and upper triangular structures, compressed storage formats, and a banded representation. The results consistently showed that the relative errors in the ℓ_2 norm were small, typically around 10^{-15}–10^{-16}, which confirms that our implementations are both stable and accurate when compared to MATLAB's built-in routines. This performance is consistent with expectations, since the operations involve only additions and multiplications of floating point values, and any discrepancies are attributable likely to rounding error.

A limitation of this work is that our analysis focused exclusively on ℓ_2-norm relative error. While this provides a robust measure of overall accuracy, additional insight could be gained by considering the ℓ_∞-norm. The ℓ_∞-norm highlights the largest componentwise deviation, which may be masked when averaging errors in the ℓ_2-norm. Including both perspectives would give a more complete picture of error behavior across problem sizes and matrix structures. Another limitation is that we restricted testing to random matrices generated from uniform and normal distributions. It would be useful to expand the experiments to structured or adversarial test cases to better probe the limits of each storage scheme. While I would have liked to implement testing of edge cases, to help highlight potential failure points, I unfortunately was unable to include such an analysis due to time constraints.

Future work could therefore be done to test the robustness and overall error of all the routines, by testing edge cases, different distributions for random matrices, and analyzing errors under different norms.

8 Disclaimer

Chat-GPT and MATLAB's built in COPILOT were both used as aids for debugging some issues with code as well as helping to improve formatting where my LaTeXskills are lacking. The bulk of all work is completed and understood on my own, as well as with some discussion with other classmates, including but not limited to: Scott Payne, Yunxi Yang and Ruojun Wang.

A Figures

This section collects all plots generated by the driver.

A.1 Lv (unit lower) routines

Routine 3: `Lvmult_row` (row-oriented inner product)

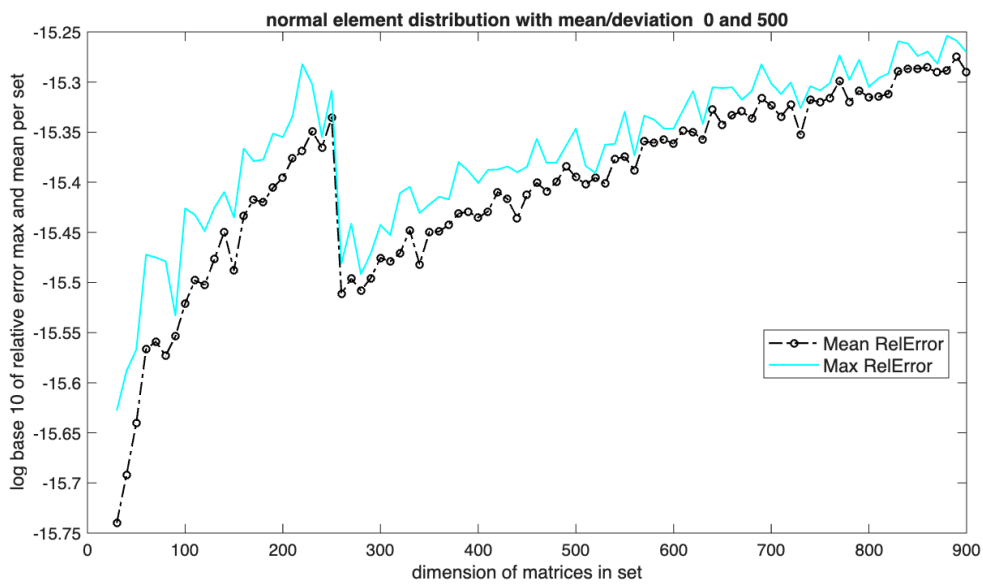

Figure 1: Routine 3 (`Lvmult_row`): Relative error behavior vs. n and summary statistics across trials.

Routine 4: `Lvmult_col` (column-oriented vector triad)

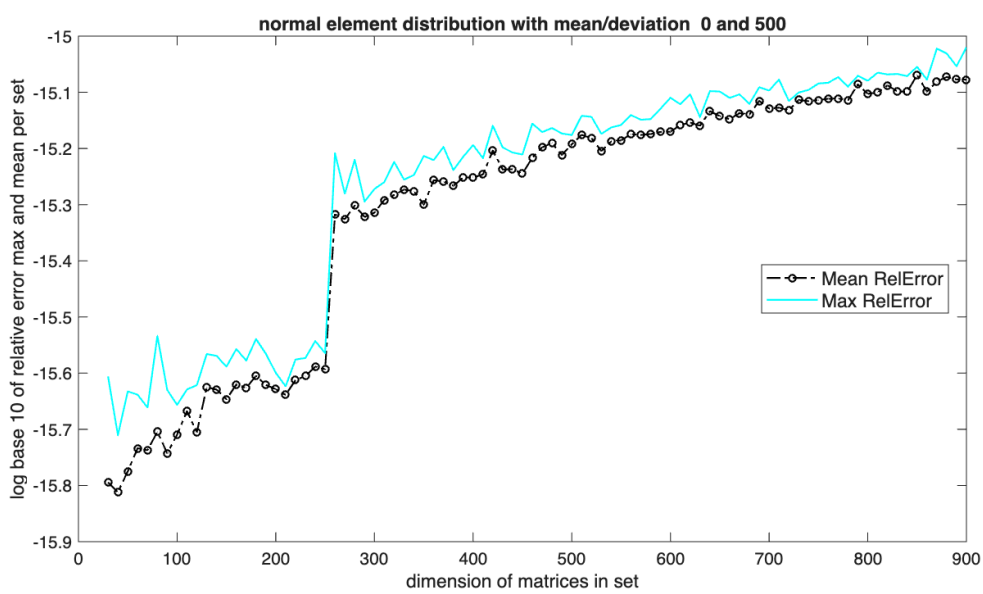

Figure 2: Routine 4 (`Lvmult_col`): Relative error behavior vs. n and summary statistics across trials.

Routine 7: `Lvmult_compcol` **(compressed column, 1D)**

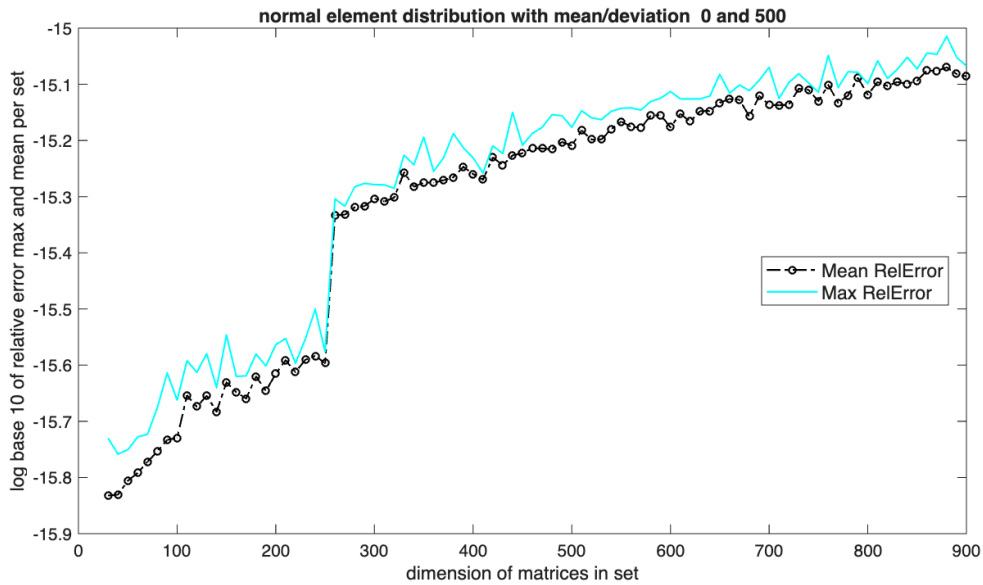

Figure 3: Routine 7 (`Lvmult_compcol`): Relative error behavior vs. n for packed strict-lower storage.

Routine 8: `Lvmult_banded` **(two subdiagonals)**

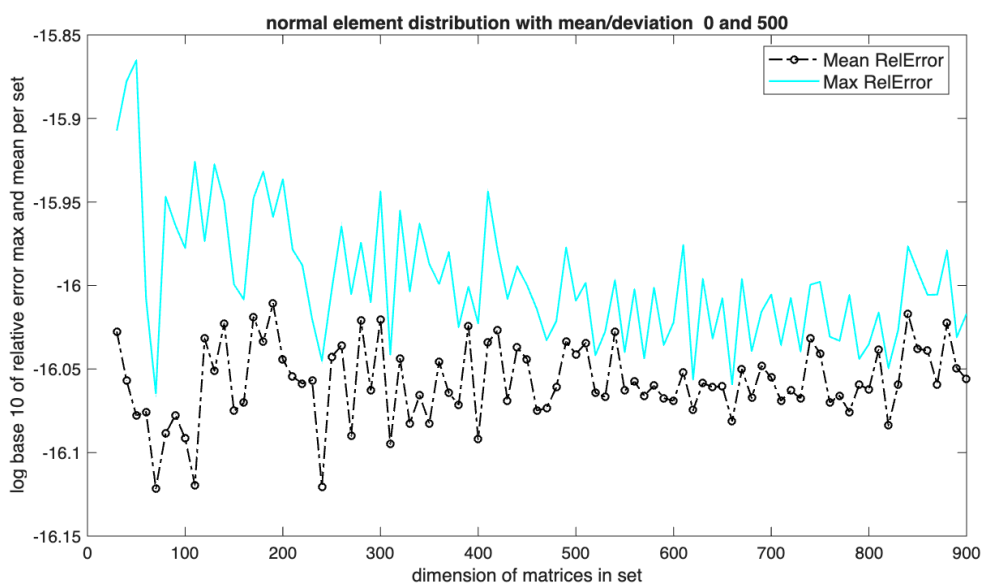

Figure 4: Routine 8 (`Lvmult_banded`): Relative error behavior vs. n for banded unit-lower L (bands -1 and -2).

Routine 0: `LUmult_outer` (outer product, stored L/U in one 2-D array)

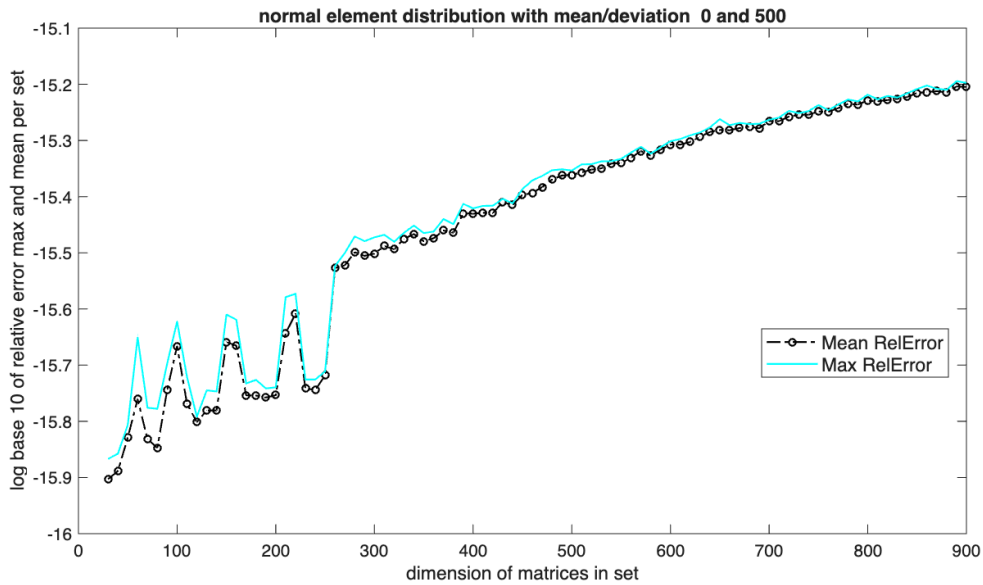

Figure 5: Routine 0 (`LUmult_outer`): Relative error behavior vs. n for $A = L\,U$ computed without unpacking.

A.2 Uv (given for validation)

Routine 5: `Uvmult_row` (row-oriented inner product)

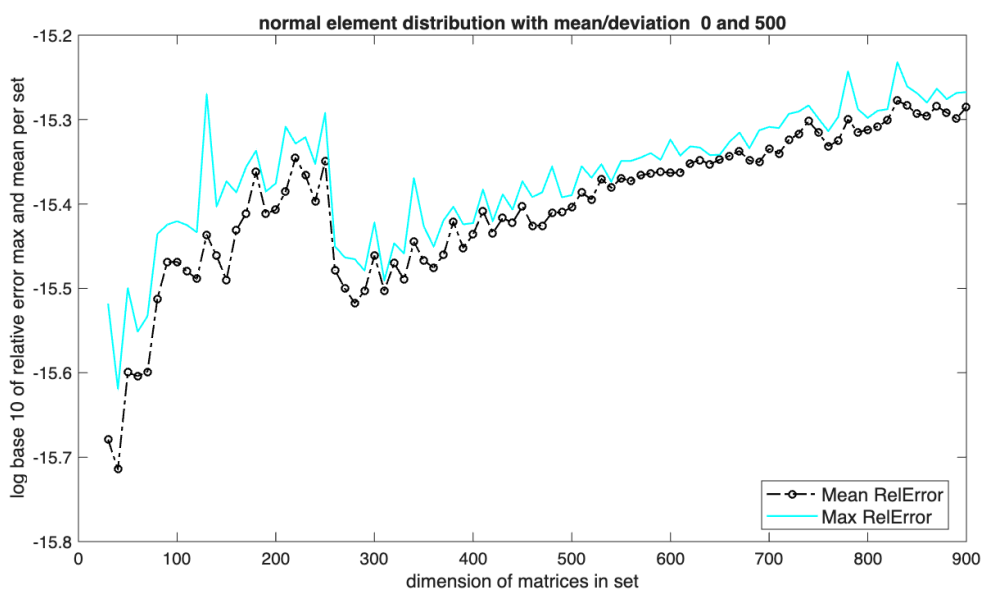

Figure 6: Routine 5 (`Uvmult_row`): Relative error behavior vs. n (given routine, included for comparison).

Routine 6: `Uvmult_col` (column-oriented vector triad)

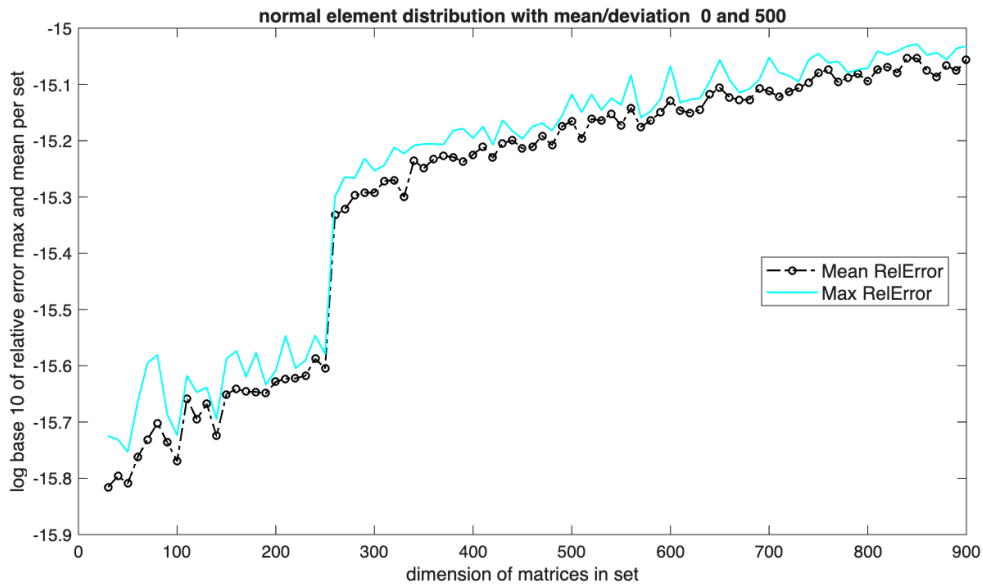

Figure 7: Routine 6 (`Uvmult_col`): Relative error behavior vs. n (given routine, included for comparison).

B Driver Script

```
%
% This is a test driver for the
% testing the multiplication subroutines
% (and their different versions)
%
% Lv-->z,  Uv-->z, LU-->A
%
%
% The actions of this routine are controlled
% by the parameters set at the beginning.
%
% Tests are run for a specifed subroutine
% for a range of values of the dimension n
% (the outer loop of code)
%
% For each value of n matrices L and U
% are generated along with a vector v.
% (the main inner loop of code)
% This can be done based on a custom
% specification of these matrices as a function
% of n (which the user must place in the code)
% or a random number generator based matrix
% with some structural constraints imposed.
% Depending on the subroutine tested,
% L and v, or U and v, or L and U will be
% used first as Matlab objects to use
% Matlab's library and commands to determine
% the "true" answer for comparison to the
% computed answer.
% The matrices and vectors required are then
% placed in the data structures as required
```

```
% by the assignment, the test is run,
% the error information collected.
%
% During this loop, if desired, detailed
% information on the results of each test problem
% can be output. Or, a summary (mean and max) of the
% relative error can be output for each set of tests
% with the same n. A third output option collects the
% information and plots the means and maximums of each set of
% tests with the same n vs the value of n.
% In all cases at the end the maximum value of the relative
% error over all tests is printed.
%

%
% The arrays M and Acomp,  are not n by n in order
% to have a code similar to that used in an
% when a range of problems would use the same
% data structure or when other information
% would be stored in M and Acomp in addition
% to L, U, and L*U. The same is true for the
% vectors v, zcomp, and ztrue.
%


%
% These set the dimensions of the  data structures.
% All problems tested should have n smaller than the smallest of these.
Mrdim=100;
Mcdim=120;
Acomprdim=150;
Acompcdim=110;
zdim=150;
vdim=110;

%
% set range of matrix dimensions and number
% of tests per dimension
%
nmin=30;
nmax=100;
ndelta=1;
ntests=5;

%
% Indicate the subroutine to be tested by commenting out those not desired.
%
% routinenum = 0; % L*U row-oriented outer product algorithm          <<-- Use
% routinenum = 1; % L*U row-oriented inner product algorithm          <<-- Wont Use
% routinenum = 2; % L*U column-oriented vector triad  algorithm       <<-- Wont Use
% routinenum = 3; % L*v row-oriented inner product algorithm          <<-- Use
% routinenum = 4; % L*v column-oriented vector triad  algorithm       <<-- Use
% routinenum = 5; % U*v row-oriented inner product algorithm          <<-- Given
% routinenum = 6; % U*v column-oriented vector triad  algorithm       <<-- Given
% routinenum = 7; % L*v compressed-column (1D) vector triad algorithm <<-- Added
% routinenum = 8; % L*v banded multiplication algorithm               <<-- Added
%
```

```
% Set the manner of matrix and vector generation by commenting out those not desired.
%
%problemtype = 1; % User custom manner. Add the code at the appropirate place below.
%problemtype = 2; % Uniform random numbers on the interval [dmin,dmax].
                 % Note that this interval's endpoints can be set to include both signs
                 % or have a fixed one based on the signs of the endpoints.
                 % this does not affect the signs of the 1's on the diagonal of L.
%dmin=-10.0;
%dmax=10.0;
problemtype = 3; % Normal random numbers with specified mean and standard deviation.
tempmean=0.0;
tempstdev=500.0;

%-----------------------------------------------------------------------------------------
% Set output type by commenting out those not desired.
%
 outputtype = 1; % norm information listed for every test problem
 %outputtype = 2; % only mean/max norm information listed for each set with fixed n
 %outputtype = 3; % mean/max norm vs n plotted (the arrays are available for
                % examination after driver runs in maxforthisn and meanforthisn arrays
%-----------------------------------------------------------------------------------------

%
% There are no parameters for the user to set below this point
% except the problemtype = 1 code to set the the manner in which
% the problem is generated, i.e., the set of numbers from which
% L and U are selected and the vector v.
%

%
% initialize max relative error for all  tests of all sizes
%
maxrelerror = 0.0;

%
% Output Label of Tests to the Matlab Terminal
%
if routinenum == 0
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'A = L*U Outer Product Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
elseif routinenum == 1
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'A = L*U Row Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
elseif routinenum == 2
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'A = L*U Column Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
elseif routinenum == 3
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'z = L*v Row  Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
elseif routinenum == 4
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'z = L*v Column  Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
```

```
elseif routinenum == 5
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'z = U*v Row  Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
elseif routinenum == 6
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'z = U*v Column  Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
elseif routinenum == 7
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'z = L*v Compressed Column Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
elseif routinenum == 8
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'z = L*v Banded Version Tests for sizes n = %g to %g by %g \n',nmin,nmax,ndelta)
  fprintf(1,'***************************************** \n')
else
  fprintf(1,'\n***************************************** \n')
  fprintf(1,'Undefined version number   = %g \n',routinenum)
  fprintf(1,'***************************************** \n')
  return; % terminate the tester and return to command prompt
end

%
% Initialize error saving vectors
%
%
Error2Norm = zeros(ntests,1);
Atrue2Norm = zeros(ntests,1);
RelError =  zeros(ntests,1);
ztrue2Norm = zeros(ntests,1);

% Initialize arrays to collect mean/max norm for every set with fixed n
nexperiments=size([nmin:ndelta:nmax]',1);
meanforthisn = zeros(nexperiments,1);
maxforthisn =  zeros(nexperiments,1);
experimentindex = 0;

%Test loop over a range of dimensions of L, U, and v
for n = nmin:ndelta:nmax

  experimentindex = experimentindex + 1;
  if (outputtype == 1) || (outputtype == 2)
    fprintf(1,'\n********************************** \n')
    fprintf(1,'Tests for size n = %g \n',n)
    fprintf(1,'********************************** \n')
  end

% loop over fixed n for ntests random problems
  for itest=1:ntests
% reinitialize data structures for a new problem with size n
    M=zeros(Mrdim,Mcdim);
    Acomp=zeros(Acomprdim,Acompcdim);
    zcomp=zeros(zdim,1);
    ztrue=zeros(zdim,1);
    v=zeros(vdim,1);
%
```

```
% Generate full expanded L and U
% along with a vector v
% and place them each of them in their
% associated work array.
%
% Note that since this is testing mulitplication routines
% there is no need to guarantee that the diagonal elements
% of U are nonzero.
%

% Save current state of random number generator.
% This can be used for reproducing the same sequence of
% test problems if needed. That is not done in this
% form of the code but it is often useful for debugging.
% the rng state can be restored with the statement rng(statesave);
% See the Matlab documentation for more details.
  statesave=rng;

% Select the type of random numbers
% and set the title of the error plot for outputtype = 3
% by commenting/uncommenting the appropriate lines

  if problemtype == 1
%
    outputtitle = "PUT YOUR CUSTOM TITLE HERE";
    Temp = ones(n,n); % . Replace with yours.
    v(1:n)=randn(n,1); % This is where you would set v all 1s
  elseif problemtype == 2
% standard uniform on (0,1) shifted to (dmin,dmax) set earlier
    strdmin=string(dmin);
    strdmax=string(dmax);
    temptitle = ["uniform distribution with shifted to (" strdmin "," strdmax ")"];
    outputtitle = join(temptitle);
    Temp = dmin.*ones(n,n) + (dmax-dmin).*(rand(n,n));
    v(1:n) = dmin.*ones(n,1) + (dmax-dmin).*(rand(n,1));
  elseif problemtype == 3
%  normal distribution with  mean and standard deviation set
%  to tempmean and tempstdev set earlier
    Temp  = tempstdev.*randn(n,n) + tempmean.*ones(n,n);
    v(1:n) = tempstdev.*randn(n,1) + tempmean.*ones(n,1);
    strmean=string(tempmean);
    strstdev=string(tempstdev);
    temptitle = ["normal element distribution with mean/deviation " strmean "and" strstdev];
    outputtitle = join(temptitle);
  else
    fprintf(1,'\n***************************************** \n')
    fprintf(1,'Undefined problem number   = %g \n',routinenum)
    fprintf(1,'***************************************** \n')
    return; % terminate the tester and return to command prompt
  end

  L = eye(n,n)+ tril(Temp,-1); % L is strict lower part of
                               % the array of random numbers
                               % and a diagonal of all 1
  U = triu(Temp);  % U is strict lower part of
                   % the array of random numbers
```

```
%
%  Generate true product of L and U using the built-in
%  Matlab matrix product operator and store the result
%  a work array Atrue not involved in your subroutines.
%

  if (routinenum == 0) || (routinenum == 1) || (routinenum == 2)
    Atrue = L*U;
  elseif (routinenum == 3) || (routinenum == 4)|| (routinenum == 7)
    ztrue(1:n) = L*v(1:n);

  elseif routinenum == 8

%----Code for extracting bands Lb1 and Lb2 from our 'banded' unit lower triangular L:----
Lb1 = diag(L, -1);       % length n-1
Lb2 = diag(L, -2);       % length n-2
LB=eye(n);               % initializes our banded matrix
for i=1:size(Lb1,1)
    LB(i+1,i)=Lb1(i);    % populates first band
end
for j=1:size(Lb2,1)
    LB(j+2,j)=Lb2(j);    % populates second band
end
%-----------------------------------------------------------------------------------------
    ztrue(1:n) = LB * v(1:n); % true calculation for Banded Matrix vector product.

  elseif (routinenum == 5) || (routinenum == 6)
    ztrue(1:n) = U*v(1:n);
  else
    fprintf(1,'\n***************************************** \n')
    fprintf(1,'Undefined routine number   = %g \n',routinenum)
    fprintf(1,'***************************************** \n')
    return; % terminate the tester and return to command prompt
  end

%
% Put L and U into the  simple compressed
% storage of L and U % in the array M.
% Note the 0s of L and U are not stored and
% the 1's of the diagonal of L are note stored in the
% work array M(mrdim,mcdim).
% The vectors v, zcomp, and ztrue are already in work arrays.
%
%     First a dense write of U, i.e., this includes elements of
%     the U array that are not actually in the mathematical matrix U
%     then overwrite 0s in U and M  with lower part of L.
%

%---------Code for constructing M from the random matrices L and U------------------

  M(1:n,1:n) = U;
  for i = 1:n-1
     M(i+1:n,i)= L(i+1:n,i);
  end
%-------------------------------------------------------------------------------------
```

```
%---------------Code for writing L as a compressed column array:--------------------
  Lc = zeros(n*(n-1)/2, 1); % initialize an array of zeros of appropriate length
  k = 1; % Indexing counter
  for j = 1:n-1           % columns from the first to the 2nd to last
    for i = j+1:n         % rows below the diagonal in column j
        Lc(k) = M(i,j);   % Assigns each entry of Lc one column of M at a time
        k = k + 1;        % Increases our index step after each entry assignment.
    end
  end
%-------------------------------------------------------------------------------


% Run test number itest for the current value of n
%
  if (routinenum == 0)
    [Acomp]=LUmult_outer(M, Acomp, n);
    ErrorMatrix = Acomp(1:n,1:n) - Atrue(1:n,1:n);
%
%   Compute the norms for this problem and save it with the others with this n
%
    Error2Norm(itest) = norm(ErrorMatrix,2);
    Atrue2Norm(itest) = norm(Atrue,2);
    RelError(itest) = Error2Norm(itest)/Atrue2Norm(itest);
%
%   update running maximum relative error over test problems for all n
%
    maxrelerror = max(maxrelerror,RelError(itest));
  elseif (routinenum == 1)
    [Acomp]=LUmult_row(M, Acomp, n);
    ErrorMatrix = Acomp(1:n,1:n) - Atrue(1:n,1:n);
%
%   Compute the norms for this problem and save it with the others with this n
%
    Error2Norm(itest) = norm(ErrorMatrix,2);
    Atrue2Norm(itest) = norm(Atrue,2);
    RelError(itest) = Error2Norm(itest)/Atrue2Norm(itest);
%
%   update running maximum relative error over test problems for all n
%
    maxrelerror = max(maxrelerror,RelError(itest));
  elseif (routinenum == 2)
    [Acomp]=LUmult_col(M, Acomp, n);
    ErrorMatrix = Acomp(1:n,1:n) - Atrue(1:n,1:n);
%
%   Compute the norms for this problem and save  with the others with this n
%
    Error2Norm(itest) = norm(ErrorMatrix,2);
    Atrue2Norm(itest) = norm(Atrue,2);
    RelError(itest) = Error2Norm(itest)/Atrue2Norm(itest);
%
%   update running maximum relative error over test problems for all n
    maxrelerror = max(maxrelerror,RelError(itest));
  elseif (routinenum == 3)
    [zcomp]=Lvmult_row(M, v, zcomp,n);
    ErrorVector = zcomp(1:n) - ztrue(1:n);
%
%   Compute the norms for this problem and save with the others with this n
```

```
%
    Error2Norm(itest) = norm(ErrorVector,2);
    ztrue2Norm(itest) = norm(ztrue,2);  % ztrue is an n element vector
    RelError(itest) = Error2Norm(itest)/ztrue2Norm(itest);

% update running maximum relative error over test problems for all n
    maxrelerror = max(maxrelerror,RelError(itest));
  elseif (routinenum == 4)
    [zcomp]=Lvmult_col(M, v, zcomp,n);
    ErrorVector = zcomp(1:n) - ztrue(1:n);
%
%   Compute the norms for this problem and save with the others with this n
%
    Error2Norm(itest) = norm(ErrorVector,2);
    ztrue2Norm(itest) = norm(ztrue,2);  % ztrue is an n element vector
    RelError(itest) = Error2Norm(itest)/ztrue2Norm(itest);
% update running maximum relative error over test problems for all n
    maxrelerror = max(maxrelerror,RelError(itest));
  elseif (routinenum == 5)
    [zcomp]=Uvmult_row(M, v, zcomp,n);
    ErrorVector = zcomp(1:n) - ztrue(1:n);
%
%   Compute the norms for this problem and save with the others with this n
%
    Error2Norm(itest) = norm(ErrorVector,2);
    ztrue2Norm(itest) = norm(ztrue,2);  % ztrue is an n element vector
    RelError(itest) = Error2Norm(itest)/ztrue2Norm(itest);
%   update running maximum relative error over test problems for all n
    maxrelerror = max(maxrelerror,RelError(itest));
  elseif (routinenum == 6)
    [zcomp]=Uvmult_col(M, v, zcomp,n);
    ErrorVector = zcomp(1:n) - ztrue(1:n);
%
%   Compute the norms for this problem and save with the others with this n
%
    Error2Norm(itest) = norm(ErrorVector,2);
    ztrue2Norm(itest) = norm(ztrue,2);  % ztrue is an n element vector
    RelError(itest) = Error2Norm(itest)/ztrue2Norm(itest);
% update running maximum relative error over test problems for all n
    maxrelerror = max(maxrelerror,RelError(itest));

  elseif (routinenum == 7)
    [zcomp]=Lvmult_compcol(Lc, v, zcomp,n);
    ErrorVector = zcomp(1:n) - ztrue(1:n);
%
%   Compute the norms for this problem and save with the others with this n
%
    Error2Norm(itest) = norm(ErrorVector,2);
    ztrue2Norm(itest) = norm(ztrue,2);  % ztrue is an n element vector
    RelError(itest) = Error2Norm(itest)/ztrue2Norm(itest);
% update running maximum relative error over test problems for all n
    maxrelerror = max(maxrelerror,RelError(itest));

    elseif (routinenum == 8)
    [zcomp]=Lvmult_banded(Lb1, Lb2, v, zcomp, n);
    ErrorVector = zcomp(1:n) - ztrue(1:n);
%
```

```
%   Compute the norms for this problem and save with the others with this n
%
    Error2Norm(itest) = norm(ErrorVector,2);
    ztrue2Norm(itest) = norm(ztrue,2);  % ztrue is an n element vector
    RelError(itest) = Error2Norm(itest)/ztrue2Norm(itest);
% update running maximum relative error over test problems for all n
    maxrelerror = max(maxrelerror,RelError(itest));

  else
    fprintf(1,'\n***************************************** \n')
    fprintf(1,'Undefined routine number   = %g \n',routinenum)
    fprintf(1,'***************************************** \n')
    return; % terminate the tester and return to command prompt
  end

  end % end of itest loop

if outputtype == 1
  for iouts = 1:ntests
   if (routinenum == 0) || (routinenum == 1) || (routinenum == 2)
    fprintf(1,'n = %g Error 2-norm = %5.2e  Atrue 2-norm = %5.2e Relative Error = %5.2e\n',n,Error2Norm(iouts), At
   else
    fprintf(1,'n = %g Error 2-norm = %5.2e  ztrue 2-norm = %5.2e Relative Error = %5.2e\n',n,Error2Norm(iouts), zt
   end % end of if
  end % output loop
end %  end outputtype ==1 if
meanforthisn(experimentindex) = mean(RelError(1:ntests));
maxforthisn(experimentindex) =  max(RelError(1:ntests));

if outputtype == 2
 fprintf(1,'For %g problems with n = %g  Mean Relative Error = %5.2e Maximum Relative Error = %5.2e\n',ntests,n, m
end

end % end of n loop

% print overall maximum relative error
fprintf(1,'\n\n\nMaximum Relative Error over all tests of all sizes = %5.2e\n',maxrelerror);

if outputtype == 3
% protect against -infinity in log vectors
tempmean(1:nexperiments)=max(-17,log10(meanforthisn(1:nexperiments)));
tempmax(1:nexperiments)=max(-17,log10(maxforthisn(1:nexperiments)));
plot([nmin:ndelta:nmax],tempmean(1:nexperiments),'-.k')  % plot
hold on
plot([nmin:ndelta:nmax],tempmax(1:nexperiments),'-r')  % plot
xlabel('dimension of matrices in set')
ylabel('log base 10 of relative error max and mean per set')
title(outputtitle);

% debug printing of mean and max data
% fprintf(1,'\n\n\nMean Relative Errors vs n = %5.2e\n',meanforthisn(1:nexperiments));
% fprintf(1,'\n\n\nMaximum  Relative Errors vs n = %5.2e\n',maxforthisn(1:nexperiments));
hold off
end
```